

SAE5.01 – Concevoir, réaliser et présenter une solution technique

Formation par apprentissage

RT3-FA-A1

20 novembre 2025

Créé par : Indira LUIS / Imène SAID ERRAHMANI / Loïck LEMEN

SOMMAIRE

Table des matières

PRESENTATION DU PROJET.....	2
MISE EN PLACE DE L'ENVIRONNEMENT.....	5
PLAN D'ADRESSAGE :	5
INSTALLATION DES OUTILS.....	6
CONFIGURATION DU SERVEUR RADIUS	9
DOCKER ET ORCHESTRATION DE L'INFRASTRUCTURE	13
CONFIGURATION DES IMAGES DOCKER.....	14
STRUCTURE DU DOSSIER IMAGES-DOCKER.....	14
IMAGES ET FONCTIONNALITES.....	14
ACCES GRAPHIQUE VIA VNC ET NOVNC	15
BUILD ET DEPLOIEMENT DES IMAGES	16
PERSONNALISATION D'UNE IMAGE DOCKER	16
DEVELOPPEMENT DE L'APPLICATION WEB (FLASK).....	19
MODULES UTILISES	19
FONCTIONNEMENT DU FLASK.....	20
BACKEND ET FRONTEND FLASK.....	21
MISE EN PLACE DE LA BASE DE DONNEES	22
MISE EN PLACE DE LA BASE DE DONNEES.....	22
ACCES A LA BASE VIA PHPMYADMIN.....	23
STRUCTURE DE LA BASE DE DONNEES	23
SECURITE DE LA PLATEFORME	25
SECURISATION DES DONNEES SENSIBLES	25
MISE EN PLACE DU CHIFFREMENT HTTPS AVEC NGINX.....	26
PROCEDURES D'UTILISATION	29
MONITORING DE L'ADMIN	30
CONCLUSION.....	31
MOTS DE PASSE UTILISES	32

Présentation du projet

Quels sont les objectifs ?

Dans le cadre de la SAE 501 – Mise en place d’une infrastructure de TP basée sur des conteneurs, ce projet a pour objectif de **concevoir, déployer et sécuriser** une plateforme permettant aux étudiants d’accéder à différents environnements pédagogiques virtualisés. L’idée centrale est de fournir une solution moderne, flexible et facilement maintenable pour héberger des machines de TP à la demande, en s’appuyant sur des **technologies de conteneurisation et d’orchestration**.

Le projet s’inscrit dans une démarche d’industrialisation des environnements de formation : au lieu de machines virtuelles lourdes, chaque environnement est désormais isolé, répliquable et modulable grâce aux conteneurs Docker. L’utilisation d’un orchestrateur et d’une interface web simplifie l’accès des utilisateurs et permet une administration centralisée.

Ainsi, la plateforme doit répondre aux besoins suivants :

- Déployer des **conteneurs personnalisés** (Ubuntu Desktop, Kali Linux, Ubuntu Server, Wireshark...);
- Offrir une interface simple permettant aux utilisateurs de **gérer leurs environnements** ;
- Assurer une **isolation** correcte des environnements et un suivi des actions ;
- Intégrer une **base de données** pour gérer dynamiquement les conteneurs liés à chaque utilisateur ;
- Proposer une architecture techniquement **fiable, sécurisée et documentée**.

Architecture générale de la solution :

La plateforme repose entièrement sur Docker et intègre plusieurs composants logiciels permettant de fournir une infrastructure complète de TP :

- **Docker & Docker Compose** pour la gestion des conteneurs et services.
- **Flask** comme backend et frontend principal, jouant le rôle de couche applicative et interface avec le moteur Docker.
- **MariaDB** gère les associations utilisateur/conteneurs.
- **Portainer** comme outil d’administration avancée des conteneurs (interface technique).
- **PHPMyAdmin** pour l’administration de la base de données.
- **Images Docker personnalisées** correspondant aux environnements de TP : Kali Linux, Ubuntu Desktop, Ubuntu Server, Ubuntu Desktop + Wireshark.

Ces composants sont orchestrés via un fichier **docker-compose.yml** permettant la mise en place automatique de l’infrastructure.

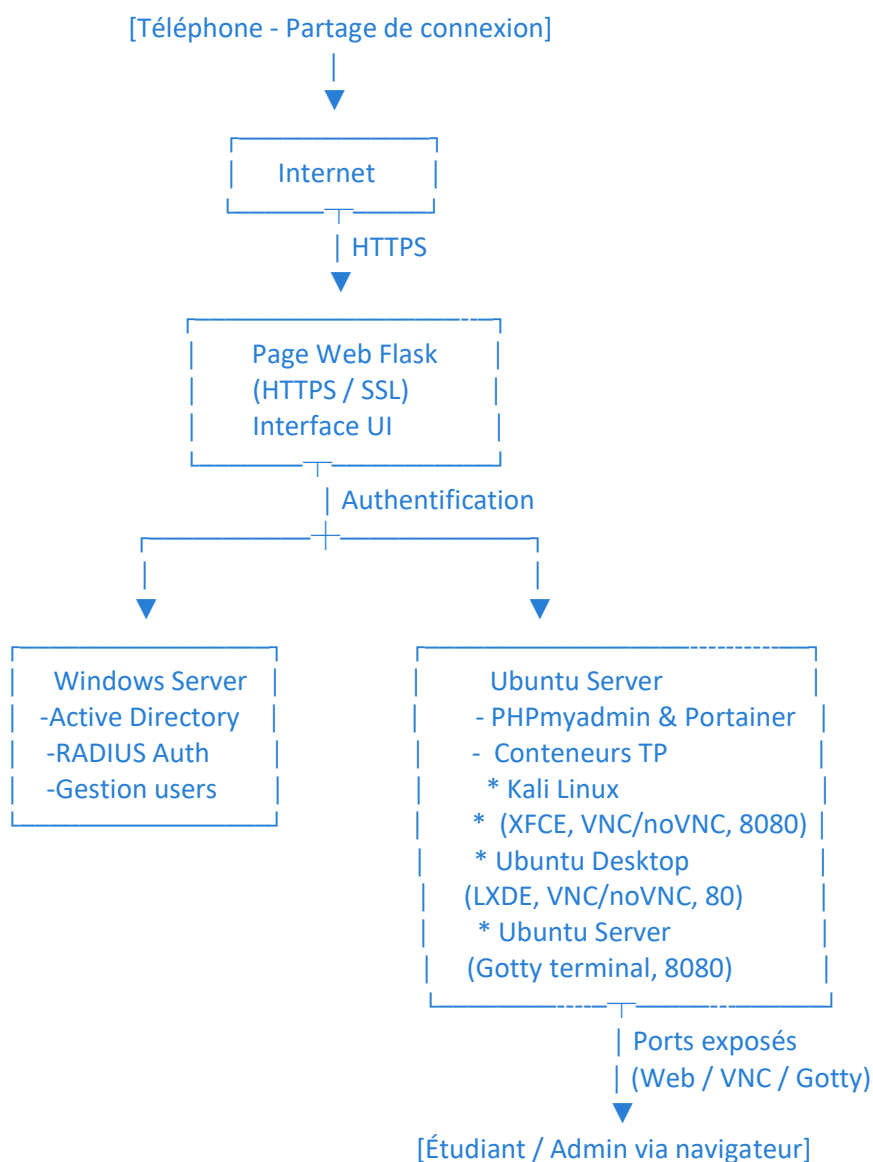
Fonctionnement global :

L'application Flask constitue le cœur de la plateforme :

- Elle gère l'authentification des utilisateurs via un serveur RADIUS externe ;
- Elle permet le déploiement à la demande de conteneurs via l'API Docker ;
- Elle offre un tableau de bord dynamique affichant les templates disponibles et les conteneurs déjà déployés ;
- Elle intègre des fonctions d'administration : création/suppression d'utilisateurs Windows via backend Flask + script PowerShell, monitoring des conteneurs...

Côté sécurité, l'isolement est assuré par les conteneurs, la gestion des utilisateurs par RADIUS, et l'accès HTTPS obligatoire pour accès à l'interface web.

Architecture du projet SAE501 :



Arborescence principale du projet SAE501 :

```
SAE501
├── app.py
├── backend
│   ├── init_db.py
│   ├── instance
│   │   └── users.db
│   ├── __pycache__
│   │   ├── app.cpython-310.pyc
│   │   └── models.cpython-310.pyc
├── creation_cle.py
├── docker-compose.yml
├── Dockerfile
├── frontend
│   ├── dashboard.html
│   └── js
│       └── main.js
├── genere_encode.py
├── images-docker
│   ├── kali
│   │   ├── Dockerfile
│   │   ├── kali_data
│   │   └── startup.sh
│   ├── ubuntu
│   │   └── Dockerfile
│   ├── ubuntu-server
│   │   └── Dockerfile
│   └── ubuntu-wireshark
│       └── Dockerfile
├── __pycache__
│   ├── app.cpython-310.pyc
│   └── models.cpython-310.pyc
├── static
│   └── css
│       └── style.css
├── templates
│   ├── create_user.html
│   ├── dashboard.html
│   └── index.html
├── user_containers.sql
└── vm.sh
```

Mise en place de l'environnement

Plan d'adressage :

MACHINE VIRTUELLE	ADRESSE	PASSERELLE
Windows Server (AD + RADIUS)	172.20.10.8/16	172.20.10.1
Ubuntu	172.20.10.9/16	172.20.10.1

Tableau des ports utilisés lors de la SAE :

Service / Conteneur	Port interne	Port externe	Description / Utilité
Flask (Backend)	5000	5000	Application web principale
phpMyAdmin	80	8080	Interface web d'administration de la base
Portainer.io	9000	9000	Interface de gestion conteneurs
Nginx (reverse proxy HTTPS)		80 / 443	Redirection HTTP → HTTPS + proxy pour Flask

⚠ Remarques importantes

Pour les machines utilisateur (Kali, Ubuntu Desktop...), les ports sont « assignés automatiquement » :

```
ports={ "8080/tcp": None }
```

Ou pour les images basées sur port 80 :

```
ports={ "80/tcp": None }
```

Cela signifie que **Docker choisit automatiquement un port libre sur l'hôte**, ce qui évite les conflits lorsqu'un utilisateur lance plusieurs conteneurs.

Installation des outils

Dans un premier temps, plusieurs outils indispensables ont été installés sur les différentes machines virtuelles afin de préparer l'environnement de travail. Les manipulations ci-dessous ont été réalisées principalement sur la machine **Ubuntu** (serveur Docker/Flask) ainsi que sur le **Windows Server** destiné à accueillir Active Directory et RADIUS.

- **Installation de VMware et des additions invitées :**

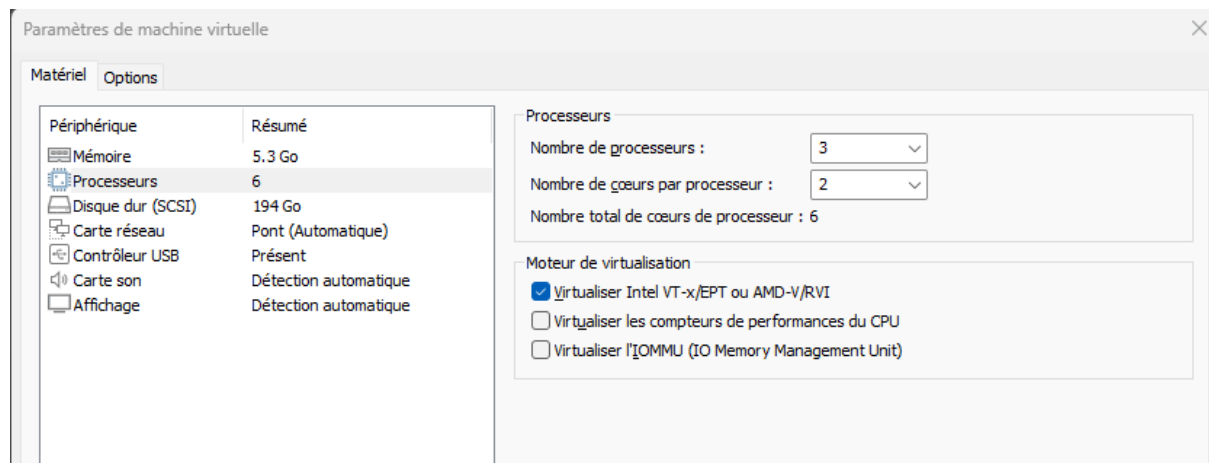
Après la création des machines virtuelles, il a été nécessaire d'installer les **VMware Tools** afin d'assurer une meilleure intégration entre l'hôte et les VM (partage presse-papier, redimensionnement automatique, optimisation graphique).

Sur la machine Ubuntu, les additions invitées ont été installées via :

```
sudo apt update  
sudo apt install open-vm-tools open-vm-tools-desktop -y
```

- **Activer la virtualisation Intel VT sur la machine virtuelle Ubuntu :**

Par la suite, la virtualisation **Intel VT** a été activée sur la machine virtuelle **Ubuntu**. Cette configuration était requise pour autoriser la mise en place d'une machine virtuelle supplémentaire dédiée au futur **conteneur serveur Windows**. L'activation d'Intel VT assure la compatibilité avec la virtualisation imbriquée, permettant ainsi d'exécuter correctement un environnement Windows Server au sein de l'infrastructure déjà virtualisée.



• Installation de Docker et Docker Compose (version 2) :

Docker constitue la base de la plateforme de virtualisation de la SAE. L'installation s'est effectuée directement via les dépôts Ubuntu :

```
sudo apt update
sudo apt install docker.io -y
```

Choix de Docker Compose version 2

Initialement, une installation de **Docker Compose version 1** avait été tentée. Cependant, cette version posait plusieurs problèmes de compatibilité avec les services utilisés dans le projet, notamment :

- Impossibilité de déployer certains conteneurs avec les nouvelles syntaxes du fichier docker-compose.yml
- Incompatibilités avec Portainer et certaines API Docker récentes
- Absence de fonctionnalités présentes uniquement dans Compose v2

En raison de ces limitations, **la version 1 ne permettait pas le bon fonctionnement de l'infrastructure**, ce qui nous a conduits à installer **Docker Compose version 2**, plus moderne, mieux supportée, et parfaitement compatible avec les besoins du projet.

L'installation du plugin Compose v2 s'est faite manuellement :

```
mkdir -p /root/.docker/cli-plugins
curl -SL \
  https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-
$(uname -m) \
  -o /root/.docker/cli-plugins/docker-compose

chmod +x /root/.docker/cli-plugins/docker-compose
```

• Activation du serveur SSH sous Windows Server (machine RADIUS) :

Pour permettre l'exécution de scripts d'administration (ex : création automatique d'utilisateur AD), nous avons installé et activé le serveur **OpenSSH** sur la machine Windows Server.

Les commandes suivantes ont été exécutées dans PowerShell en mode administrateur :

```
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
Start-Service sshd
Set-Service -Name sshd -StartupType 'Automatic'
```

Cela permet d'automatiser la création et suppression d'utilisateurs AD via des connexions SSH depuis la machine Ubuntu.

- **Préparation de l'environnement Python :**

Certaines fonctionnalités du backend Flask requièrent la gestion sécurisée de variables sensibles (mots de passe, clés de chiffrement).

Pour cela, plusieurs bibliothèques Python ont été installées :

```
sudo apt install python3  
sudo apt install python3-pip  
pip install python-dotenv cryptography
```

- **python-dotenv** permet de charger des variables depuis un fichier .env
- **cryptography** permet de chiffrer les données sensibles (mot de passe MariaDB, clés d'accès, etc...

Configuration du serveur RADIUS

Dans la continuité de la mise en place de l'infrastructure, une étape essentielle a consisté à configurer un serveur RADIUS sur la machine Windows Server. Ce service joue un rôle central dans le projet : il permet d'assurer l'authentification des utilisateurs et des machines, notamment pour les connexions réseau sécurisées, en s'appuyant directement sur Active Directory. Le choix d'un serveur RADIUS s'impose donc pour bénéficier d'un contrôle d'accès fiable, centralisé et compatible avec les différents services utilisés dans la SAE.

• Installation d'Active Directory et création de la forêt :

Avant de pouvoir configurer le serveur RADIUS, la machine Windows Server a été promue en contrôleur de domaine.

L'installation du rôle Active Directory Domain Services (AD DS) a permis de créer une nouvelle forêt, constituant la base de toute la gestion des utilisateurs et groupes nécessaires au fonctionnement de l'infrastructure.

Cette forêt devient ainsi le point central de l'authentification, sur lequel le serveur RADIUS va venir s'appuyer pour valider les identités.

• Créations du groupe et des utilisateurs :

Une fois la forêt déployée, un groupe Active Directory a été créé afin de pouvoir les intégrer dans le radius. Cette structuration préalable facilite la configuration ultérieure des politiques NPS, qui se basent en grande partie sur l'appartenance des utilisateurs à certains groupes AD.

• Configuration des stratégies NPS :

Une fois le rôle installé, plusieurs paramètres ont été mis en place afin d'assurer un contrôle strict de l'accès :

- **Déclaration des clients RADIUS** : enregistrement de la machine Ubuntu autorisée à communiquer avec le serveur.
- **Définition des politiques de connexion** : règles déterminant si un utilisateur est autorisé ou non à se connecter, selon son groupe.
- **Configuration des méthodes d'authentification** (PEAP, EAP, MS-CHAPv2, etc.).

• Automatisation via des scripts PowerShell :

Pour simplifier la gestion des utilisateurs Active Directory, des **scripts PowerShell** ont été ajoutés afin d'automatiser la création et la suppression des comptes.

Create_user.ps1 permet de créer un compte utilisateur dans l'OU par défaut, d'appliquer une configuration sécurisée du mot de passe, puis d'ajouter automatiquement l'utilisateur dans un groupe Active Directory spécifique.

```

param(
    [Parameter(Mandatory=$true)]
    [string]$UserName,
    [Parameter(Mandatory=$true)]
    [string]$Password
)
# Convertir le password
$SecurePassword = ConvertTo-SecureString $Password -AsPlainText -Force
# Affichage = identique au UserName
$DisplayName = $UserName
# OU par défaut
$OU = "CN=Users,DC=sae,DC=lan"
# Groupe cible
$Group = "sae"
# Création du compte utilisateur
New-ADUser `
    -Name $DisplayName `
    -SamAccountName $UserName `
    -UserPrincipalName "$UserName@sae.lan" `
    -DisplayName $DisplayName `
    -Path $OU `
    -AccountPassword $SecurePassword `
    -Enabled $true `
    -ChangePasswordAtLogon $false `
    -PasswordNeverExpires $true `
    -CannotChangePassword $true
Add-ADGroupMember -Identity $Group -Members $UserName
Write-Host " Utilisateur '$UserName' créé et ajouté au groupe '$Group'"

```

Delete_user.ps1 assure la suppression complète d'un compte AD tout en retirant l'utilisateur du groupe associé.

Une vérification préalable évite toute erreur si l'utilisateur n'existe pas.

```

param(
    [Parameter(Mandatory=$true)]
    [string]$UserName
)
# Groupe cible
$Group = "sae"
# Vérifie si l'utilisateur existe réellement dans l'AD
$User = Get-ADUser -Filter "SamAccountName -eq '$UserName'" -ErrorAction SilentlyContinue
if (-not $User) {
    Write-Host " L'utilisateur '$UserName' n'existe pas dans Active Directory." -ForegroundColor Red
    exit
}
# Retirer du groupe si présent (évite les erreurs)
try {
    Remove-ADGroupMember -Identity $Group -Members $UserName -Confirm:$false -ErrorAction SilentlyContinue
} catch {}
# Supprimer complètement l'utilisateur
Remove-ADUser -Identity $UserName -Confirm:$false
Write-Host " Utilisateur '$UserName' supprimé complètement de l'AD." -ForegroundColor Green

```

Grâce à l'activation du serveur SSH sur Windows Server, ces scripts ont pu être exécutés à distance depuis la machine Ubuntu, notamment par l'application Flask.

- **Liaison au client et tests d'authentications :**

Pour finaliser la configuration, le client a été relié au serveur RADIUS en paramétrant :

- **L'adresse du serveur NPS**
- **La clé secrète partagée**
- **La méthode d'authentification utilisée**

Propriétés de imene

Paramètres Avancé

☒ Activer ce client RADIUS

☐ Sélectionner un modèle existant :

Nom et adresse

Nom convivial :
imene

Adresse (IP ou DNS) :
172.20.10.9 Vérifier...

Secret partagé

Sélectionnez un modèle de secrets partagés existant :
Aucun

Pour taper manuellement un secret partagé, cliquez sur Manuel. Pour générer automatiquement un secret partagé, cliquez sur Générer. Vous devez configurer le client RADIUS avec le même secret partagé entré ici. Les secrets partagés respectent la casse.

☒ Manuel ☐ Générer

Secret partagé :
.....

Confirmez le secret partagé :
.....

OK Annuler Appliquer

Des tests de connexion ont ensuite confirmé le bon fonctionnement de l'ensemble : le client envoie bien ses requêtes au serveur RADIUS, qui consulte l'Active Directory et applique les règles définies dans NPS.

Docker et orchestration de l'infrastructure

Outil de gestion multi-conteneurs utilisé : Docker Compose (version 2)

Nous avons volontairement choisi de **ne pas utiliser d'orchestrateur** car notre plateforme fonctionne sur une seule machine hôte pour un usage pédagogique. L'outil retenu pour ce projet est **Docker Compose version 2**. Ce choix répond à un objectif essentiel : disposer d'un outil simple, efficace et parfaitement adapté à une infrastructure mononœud destinée à de la formation et à des environnements de test.

Docker Compose V2 permet de déployer et de gérer l'ensemble des services nécessaires à la plateforme (Flask, MariaDB, Portainer, conteneurs des TP) au sein d'un environnement reproductible. Il assure automatiquement la création des réseaux, la gestion des volumes persistants et le lancement des conteneurs dans le bon ordre, garantissant ainsi une infrastructure stable et cohérente.

Pourquoi ne pas utiliser Kubernetes ?

Kubernetes est un orchestrateur extrêmement puissant, mais il est conçu pour des environnements **complexes et distribués** :

- **Auto-scaling** : Kubernetes ajuste automatiquement le nombre de conteneurs en fonction de la charge. Cette fonctionnalité est inutile ici, car les conteneurs sont lancés manuellement par les étudiants et non en fonction du trafic.
- **Haute disponibilité** : Kubernetes gère la réplication automatique des services pour assurer une continuité en cas de panne d'un nœud. Notre plateforme fonctionne sur un **unique serveur** : aucun besoin de réplication multi-nœud.
- **Gestion avancée** : Ingress, services mesh, controllers... autant de composants puissants mais largement disproportionnés pour une infrastructure pédagogique.

Kubernetes nécessite :

- Des ressources matérielles plus élevées
- Une configuration longue
- Une courbe d'apprentissage importante
- Souvent plusieurs nœuds pour exploiter pleinement ses capacités.

Pourquoi ne pas utiliser Docker Swarm ?

Docker Swarm est plus simple que Kubernetes mais reste un orchestrateur orienté **clusters distribués** :

- Il gère des **services répliqués**, utiles pour les applications web à fort trafic, mais inutiles pour une plateforme où les conteneurs sont **indépendants et lancés à la demande**.
- Swarm est pensé pour des environnements **multi-nœuds**, alors que l'infrastructure du projet est intégralement **mononœud**.
- Swarm apporterait une complexité supplémentaire (gestion des managers/workers, du consensus Raft) sans avantage réel pour ce type d'utilisation.

Configuration des images Docker

Pour la SAE 501, plusieurs images Docker personnalisées ont été créées afin de fournir différents environnements de TP. Ces images sont stockées dans le **dossier images-docker**, chaque sous-dossier correspondant à une image spécifique avec son **Dockerfile** et, si nécessaire, des scripts de démarrage (**startup.sh**).

L'objectif est de disposer de conteneurs prêts à l'emploi, sécurisés, et configurés pour l'accès distant via VNC/noVNC

Structure du dossier images-docker

Pour centraliser la gestion des environnements pédagogiques, chaque image Docker est stockée dans un sous-dossier spécifique à son type. L'arborescence principale est la suivante :

```
images-docker/  
├── kali/  
│   ├── Dockerfile  
│   └── startup.sh  
├── ubuntu/  
│   └── Dockerfile  
├── ubuntu-server/  
│   └── Dockerfile  
└── ubuntu-wireshark/  
    └── Dockerfile
```

Chaque sous-dossier contient le Dockerfile correspondant ainsi que les scripts nécessaires au démarrage et à la configuration du conteneur.

Images et fonctionnalités

Kali Linux :

Base : image officielle kalilinux/kali-rolling

Bureau : XFCE

Services graphiques : serveur VNC (port 5900) et noVNC (port 8080)

Utilisateur standard : kali avec mot de passe progt00

Script de démarrage : startup.sh, qui :

- initialise le serveur VNC
- configure noVNC pour l'accès via navigateur
- nettoie les fichiers temporaires pour éviter les conflits

Objectif pédagogique : fournir un environnement Kali complet accessible graphiquement pour les TP de sécurité et réseau.

Ubuntu Desktop :**Base** : image dorowu/ubuntu-desktop-lxde-vnc**Services graphiques** : serveur VNC (port 5900) et noVNC (port 80)**Utilisateur standard** : administrateur avec mot de passe progt00**Objectif pédagogique** : fournir un bureau Linux léger pour les TP généraux, avec un accès web via noVNC.**Ubuntu Server :****Base** : image ubuntu:latest**Services** : Gotty (serveur web pour terminal Bash, port 8080)**Objectif pédagogique** : permettre l'accès à un terminal serveur Ubuntu via navigateur, pour les exercices nécessitant uniquement une interface CLI.**Ubuntu Desktop + Wireshark :****Base** : image dorowu/ubuntu-desktop-lxde-vnc**Logiciel supplémentaire** : Wireshark**Services graphiques** : serveur VNC (port 5900) et noVNC (port 80)**Objectif pédagogique** : offrir un poste graphique pour l'analyse réseau et la capture de paquets.**Accès graphique via VNC et noVNC**

Pour chaque conteneur graphique :

Technologie	Port interne	Port externe	Usage
VNC	5900	Dynamique	Accès via client VNC classique
NoVNC	8080 ou 80	Dynamique	Accès via navigateur web (interface HTML5)

Les ports externes des conteneurs sont alloués automatiquement par Docker pour éviter les conflits lorsque plusieurs étudiants déploient des conteneurs simultanément.

Build et déploiement des images

Avant le lancement de la plateforme, les images doivent être construites localement depuis le dossier images-docker :

```
cd images-docker/kali
docker build -t kali:latest .

cd ../ubuntu
docker build -t ubuntu-desktop:latest .

cd ../ubuntu-server
docker build -t ubuntu-server-gotty:latest .

cd ../ubuntu-wireshark
docker build -t ubuntu-desktop-wireshark:latest .
```

- Chaque commande crée une image Docker prête à être déployée via Docker Compose
- Les conteneurs graphiques sont ainsi isolés et configurés pour l'accès distant sécurisé

Personnalisation d'une image docker

Afin de rendre la plateforme plus flexible et d'adapter les environnements aux besoins spécifiques des TP, un mécanisme de **création automatisée d'images Docker personnalisées** a été mis en place. L'objectif est de permettre à un utilisateur (même sans compétences avancées en conteneurisation) de générer rapidement une nouvelle image prête à être utilisée dans la plateforme de virtualisation.

Pour atteindre cet objectif, un script dédié nommé **vm.sh** a été développé. Celui-ci guide l'utilisateur étape par étape dans la création d'un nouveau conteneur, génère automatiquement les fichiers nécessaires (Dockerfile, scripts de démarrage, dossier dédié...) et intègre ensuite l'image dans l'application Flask pour qu'elle soit immédiatement disponible dans l'interface web.

Quel est l'objectif du script vm.sh ?

Le script permet de :

- Créer automatiquement une nouvelle image Docker basée sur un modèle prédéfini
- Ajouter des logiciels supplémentaires choisis par l'utilisateur
- Générer automatiquement un Dockerfile complet et fonctionnel
- Intégrer l'image dans l'interface web sans aucune manipulation manuelle
- Reconstruire automatiquement l'infrastructure Docker pour activer la nouvelle image

Grâce à ce système, aucune connaissance avancée en création d'image Docker n'est nécessaire : **l'utilisateur répond simplement à des questions, et le script fait tout le reste.**

Étapes de fonctionnement du script

Le script suit un déroulement logique conçu pour être simple et intuitif.

1 - Choix du type de machine à créer

Dès son lancement, le script demande de choisir l'un des modèles suivants :

- **Ubuntu-server** (version serveur sans interface graphique)
- **Ubuntu-desktop** (bureau graphique basé sur LXDE)
- **Kali** (distribution Linux axée cybersécurité)

Ces trois modèles correspondent aux environnements les plus utilisés dans les travaux pratiques. Le script refuse toute mauvaise saisie, ce qui évite les erreurs dès le départ.

2 - Personnalisation simple : nom + description

L'utilisateur renseigne ensuite :

- Un **nom**, en minuscules, qui servira d'identifiant interne
- Une **description**, affichée dans l'interface de la plateforme

Ce mécanisme permet de retrouver facilement l'image dans la liste disponible.

3 - Installation de logiciels supplémentaires

Pour Ubuntu Server et Kali, le script propose d'installer librement des paquets supplémentaires :

```
nmap http iperf3 git ...
```

Il suffit d'écrire les noms séparés par des espaces. Ces paquets seront directement intégrés dans l'image Docker construite.

Remarque importante : seule l'image **Ubuntu Desktop** permet d'ajouter des options avancées via le script **nixite.sh**.

4 - Création automatique du dossier de l'image

Le script crée une arborescence dédiée :

```
images-docker/
├── nom_de_la_machine/
│   ├── Dockerfile
│   └── (startup.sh si nécessaire)
```

Cela garantit une bonne organisation du projet et évite les mélanges entre différentes images.

5 - Génération automatique du Dockerfile

En fonction du modèle choisi, le script construit automatiquement un Dockerfile :

Pour Ubuntu Server

- Installation des paquets choisis
- Ajout de Gotty, un terminal accessible via navigateur
- Lancement automatique sur le port 8080

Pour Ubuntu Desktop

- Interface graphique LXDE avec VNC/noVNC
- Installation d'outils de base (curl, vim...)
- Exécution automatique du script d'installation nixite.sh

Pour Kali

- Installation du bureau XFCE
- Installation du serveur VNC et de noVNC
- Création d'un script startup.sh pour lancer l'environnement graphique
- Ajout d'un utilisateur *kali* prêt à l'emploi

Le Dockerfile est généré entièrement par le script, ce qui évite à l'utilisateur d'écrire la moindre ligne technique.

Développement de l'application web (FLASK)

L'application Flask constitue le cœur de la plateforme SAE501. Elle sert d'interface web pour les utilisateurs, qu'il s'agisse d'étudiants ou d'administrateurs, et centralise la gestion de l'ensemble des conteneurs pédagogiques. Son rôle principal est de fournir un point d'accès unique pour visualiser l'état des conteneurs, lancer des TP, et interagir avec les différents services nécessaires au bon fonctionnement de la plateforme.

Modules utilisés

- **docker**
Permet d'interagir directement avec le moteur Docker. Il sert à créer, démarrer, arrêter et supprimer les conteneurs depuis Flask.
- **mysql.connector**
Gère la connexion à la base de données MariaDB pour écrire et lire les informations concernant les utilisateurs et les conteneurs.
- **Paramiko**
Utilisé pour établir une connexion SSH au serveur Windows Server. Il permet d'exécuter automatiquement des scripts PowerShell pour créer ou supprimer des comptes utilisateurs dans Active Directory et RADIUS.
- **radius**
Sert à valider les identifiants des utilisateurs en réalisant l'authentification auprès du serveur RADIUS.
- **cryptography.fernet**
Assure le chiffrement des informations sensibles (mots de passe, secrets, clés). Les données ne sont jamais stockées en clair dans la base de données.
- **python-dotenv**
Permet de charger les variables sensibles (mots de passe, secrets de chiffrement, identifiants BD...) depuis un fichier .env pour une configuration plus propre et sécurisée.

Fonctionnement du Flask

Flask permet aux étudiants de se connecter via leur navigateur en toute sécurité grâce à HTTPS. Les identifiants fournis sont transmis au serveur Windows, qui intègre Active Directory et le service RADIUS, pour authentification. Cette architecture assure que chaque utilisateur dispose d'un accès personnalisé et que les droits sont correctement appliqués. L'application Flask reçoit ensuite la confirmation de l'authentification et ouvre la session de l'utilisateur, qui est redirigé vers le tableau de bord.

Le tableau de bord joue un rôle central dans la supervision et l'orchestration des conteneurs. Flask utilise le SDK Python pour Docker afin de lister tous les conteneurs disponibles sur le serveur Ubuntu. Il récupère leur état, les ports exposés et les informations nécessaires pour chaque conteneur, comme le type de TP et l'utilisateur associé. Grâce à cette interface, il est possible de démarrer, arrêter ou supprimer un conteneur en un simple clic, ce qui simplifie considérablement la gestion pédagogique et réduit les erreurs liées au déploiement manuel.

Flask communique également avec la base de données MariaDB pour conserver l'historique des conteneurs et des sessions. Chaque action effectuée par un utilisateur, qu'il s'agisse du lancement d'un TP ou de la suppression d'un conteneur, est enregistrée, permettant à la plateforme de garder une traçabilité complète.

Un aspect essentiel de Flask dans cette plateforme est la gestion centralisée des utilisateurs sur Windows Server. Pour ce faire, l'application utilise le module Paramiko afin de se connecter en SSH au serveur Windows et d'exécuter des scripts PowerShell. Ces scripts permettent de créer de nouveaux utilisateurs ou de supprimer des comptes existants. Lors de la création d'un utilisateur, le script PowerShell configure automatiquement le compte dans Active Directory et l'associe au serveur RADIUS, garantissant que le nouvel utilisateur pourra se connecter à la plateforme et lancer ses conteneurs avec les droits appropriés. De la même manière, lors de la suppression d'un utilisateur, le script supprime le compte de l'Active Directory et désactive l'accès RADIUS, empêchant toute connexion non autorisée. Cette approche assure que la gestion des utilisateurs est cohérente, sécurisée et synchronisée entre la plateforme web et l'infrastructure Windows, sans intervention manuelle sur le serveur.

L'ensemble de ces interactions fait de Flask le cœur fonctionnel de la plateforme, reliant les utilisateurs, les conteneurs Docker, le serveur Windows et la base de données. Chaque action sur le tableau de bord entraîne une orchestration automatique des conteneurs, en respectant les permissions des utilisateurs et en exposant les ports nécessaires pour accéder aux services comme VNC, noVNC ou Gotty. La création et la suppression des comptes utilisateurs via Windows Server assurent que les droits d'accès sont correctement appliqués et que la sécurité de la plateforme est maintenue. Cette organisation assure un déploiement fluide et sécurisé des TP, tout en offrant aux étudiants une interface simple et intuitive pour interagir avec leur environnement pédagogique.

Backend et Frontend Flask

L'architecture de la plateforme repose sur une séparation claire entre le frontend et le backend, garantissant modularité et maintenabilité. Le frontend correspond à l'interface web que les étudiants et les administrateurs utilisent dans leur navigateur. Il est constitué de pages HTML, CSS et JavaScript, organisées dans le dossier templates pour Flask et static pour les fichiers statiques. Ces pages affichent les informations des conteneurs, les boutons pour démarrer ou arrêter un TP, ainsi que les formulaires de création ou suppression d'utilisateur. Le frontend communique avec le backend via les routes Flask, qui gèrent la logique métier et les interactions avec Docker, la base de données et le serveur Windows. Le backend, assuré par Flask, orchestre l'ensemble des services de la plateforme. Il reçoit les requêtes du frontend, exécute la logique nécessaire pour lancer, arrêter ou supprimer des conteneurs, et renvoie les informations mises à jour pour être affichées dans le navigateur. Le backend gère également la sécurité, en vérifiant les sessions utilisateurs et en chiffrant les données sensibles. Il agit comme une passerelle centralisée entre le frontend, les conteneurs Docker, la base de données MariaDB et le serveur Windows, garantissant que toutes les actions sont cohérentes et sécurisées.

Grâce à cette séparation, le frontend reste léger et responsive, offrant une expérience utilisateur intuitive, tandis que le backend prend en charge les opérations complexes d'orchestration et de sécurité. Cette organisation assure que la plateforme est facilement extensible et modulable, permettant d'ajouter de nouveaux TP, services ou fonctionnalités sans perturber le fonctionnement global.

En résumé, Flask constitue le moteur central de la plateforme SAE501. Il relie de manière sécurisée le frontend et le backend, permet de gérer les utilisateurs via Windows Server et RADIUS, orchestre les conteneurs Docker et maintient un suivi précis de toutes les actions. Le frontend offre une interface simple et intuitive, tandis que le backend prend en charge l'ensemble des traitements et la communication avec les services d'infrastructure. Ensemble, ils fournissent une plateforme stable, sécurisée et parfaitement adaptée à un usage pédagogique sur un environnement mononœud.

Mise en place de la base de données

Mise en place de la base de données

Afin de gérer la persistance des informations liées aux utilisateurs et à leurs conteneurs, l'infrastructure s'appuie sur une base de données **MariaDB**, déployée sous forme de service Docker. Cette base centralise l'association entre chaque utilisateur authentifié et les conteneurs Docker qu'il crée, ce qui permet à la plateforme d'appliquer des contrôles d'accès précis et d'offrir une vision claire de l'activité.

Choix et rôle de la base de données :

MariaDB a été retenu pour sa **légèreté**, sa **compatibilité** totale avec MySQL et sa **facilité** d'intégration dans un environnement Docker. Elle permet notamment destocker les conteneurs créés par chaque utilisateur, vérifier quels conteneurs doivent apparaître dans le tableau de bord d'un utilisateur donné, permettre à l'administrateur d'accéder à l'ensemble des conteneurs, assurer la cohérence entre l'application web et l'état réel de l'infrastructure.

La base constitue ainsi un élément central garantissant la traçabilité et l'isolation logique entre les utilisateurs.

Déploiement du service MariaDB via Docker Compose :

Le service MariaDB est intégré directement dans l'orchestrateur **Docker Compose (version 2)**. Le fichier **docker-compose.yml** définit l'image MariaDB à utiliser, les variables d'environnement (mot de passe root, nom de la base) car **il n'est pas possible de crypter les mots de passe** dans le fichier docker-compose. Il définit également un fichier d'initialisation importé automatiquement au premier démarrage.

Ce mécanisme permet de disposer d'une base immédiatement fonctionnelle et préconfigurée, sans intervention manuelle supplémentaire.

Configuration intégrée dans **docker-compose.yml** :

```
version: '3.8'

services:
  db:
    image: mariadb:10.5
    container_name: mysql
    restart: always
    volumes:
      - dbdata:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: progr00    # mot de passe root clair et fixe
      MYSQL_DATABASE: project_web_site
      MYSQL_USER: project_user        # utilisateur standard
      MYSQL_PASSWORD: progr00        # mot de passe utilisateur
```

Accès à la base via phpMyAdmin

Pour faciliter l'administration et permettre des vérifications rapides, un service **phpMyAdmin** est également déployé. Il se connecte directement au **service db**, ce qui offre une interface web accessible sur le **port 8080** du serveur hôte.

```
phpmyadmin:  
image: phpmyadmin/phpmyadmin  
container_name: phpmyadmin  
restart: always  
depends_on:  
  - db  
ports:  
  - "8080:80"  
environment:  
  PMA_HOST: db  
  MYSQL_ROOT_PASSWORD: progtr00
```

Cette interface permet notamment de vérifier l'état des tables, d'inspecter les conteneurs enregistrés ou encore de gérer la base sans outil externe.

Structure de la base de données

La base de données MariaDB utilisée par la plateforme contient une table principale : **user_containers**. Cette table permet d'assurer l'association entre les utilisateurs authentifiés et les conteneurs Docker qu'ils génèrent via l'interface web.

Table user_containers :

La table user_containers est créée automatiquement lors du déploiement initial à partir du fichier SQL suivant : **user_containers.sql**. Elle contient trois champs essentiels :

Colonne	Type	Description
ID	int(10), AUTO_INCREMENT	Identifiant unique de l'entrée
Container_id	varchar(255)	Identifiant du conteneur Docker
User	varchar(255)	Nom de l'utilisateur associé

Voici un extrait représentatif du script SQL :

```
CREATE TABLE `user_containers` (  
  `ID` int(10) NOT NULL,  
  `Container_id` varchar(255) NOT NULL,  
  `user` varchar(255) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
ALTER TABLE `user_containers`  
  ADD PRIMARY KEY (`ID`);  
  
ALTER TABLE `user_containers`  
  MODIFY `ID` int(10) NOT NULL AUTO_INCREMENT;
```

Rôle de cette table :

Cette structure permet d'assurer :

- **Le suivi des conteneurs** créés par chaque utilisateur via l'application
- **L'isolation des environnements** : chaque utilisateur ne peut gérer que ses propres conteneurs
- **La cohérence applicative** entre l'état réel de Docker et les actions côté Flask
- **La possibilité d'audit** pour l'administrateur (ex. vérification des conteneurs créés).

Exemple de fonctionnement :

```
INSERT INTO `user_containers` (`ID`, `Container_id`, `user`) VALUES  
(1, 'a625659ae26c', 'imene');
```

Cette ligne indique que l'utilisateur *imene* possède le conteneur Docker dont l'ID commence par a625659ae26c.

Sécurité de la plateforme

Sécurisation des données sensibles

La plateforme utilise un mécanisme de **chiffrement symétrique** pour protéger les informations sensibles nécessaires au fonctionnement de l'application : identifiants RADIUS, mots de passe administrateur, credentials SSH pour le serveur Windows, ainsi que les accès MySQL. L'objectif est d'éviter que ces informations **apparaissent en clair dans le code source** ou dans les conteneurs Docker.

Utilisation d'une clé secrète stockée dans un fichier .env :

Les clés utilisées pour le chiffrement sont stockées dans un fichier **.env**, accessible exclusivement par l'administrateur.

```
SECRET_KEY=zikIkFcXYTWsDNZOK9EyJVqF9j499XryPtUskXQ3iJ0=  
FLASK_SECRET_KEY=zikIkFcXYTWsDNZOK9EyJVqF9j499XryPtUskXQ3iJ0=
```

- **SECRET_KEY** : clé utilisée par **Fernet** pour chiffrer/déchiffrer les données sensibles.
- **FLASK_SECRET_KEY** : clé utilisée par Flask pour signer les cookies de session (protection contre falsification).

Le fichier .env est chargé au démarrage par :

```
load_dotenv()  
app.secret_key = os.getenv("FLASK_SECRET_KEY")  
SECRET_KEY = os.getenv("SECRET_KEY").encode()  
cipher = Fernet(SECRET_KEY)
```

Ce fichier n'est **pas intégré au dépôt Git (via .gitignore)**, empêchant toute exposition accidentelle des secrets.

BUT : seul l'administrateur peut y accéder, car sa compromission impliquerait la possibilité de déchiffrer toutes les données protégées.

Chiffrement symétrique via Fernet :

L'application utilise la bibliothèque **Cryptography** et l'algorithme **Fernet**, qui garantit :

- Chiffrement **symétrique AES-128** sécurisé
- Utilisation d'un HMAC (intégrité)
- Impossibilité de modifier ou falsifier le message sans être détecté

Les fonctions suivantes sont utilisées pour chiffrer et déchiffrer les secrets :

```
def encrypt_data(data):  
    return cipher.encrypt(data.encode()).decode()  
  
def decrypt_data(data):  
    return cipher.decrypt(data.encode()).decode()
```

Les credentials sont donc stockés dans le code **chiffré**, par exemple :

```
user = decrypt_data('gAAAAABpG0LEeOSm9AUMVIqyWvGP9AfyBsPtrsdc_CB...')  
password = decrypt_data('gAAAAABpG0LEjKb5o3xV...')
```

Ainsi, même si le fichier Python est exposé, un attaquant ne peut **rien récupérer** sans avoir la `SECRET_KEY` du fichier `.env`.

Mise en place du chiffrement HTTPS avec Nginx

Afin de sécuriser les communications entre les utilisateurs et l'application web, l'ensemble de la plateforme a été configuré pour fonctionner exclusivement en **HTTPS**. Cette couche de chiffrement est essentielle pour éviter :

- L'interception de mots de passe
- Les attaques de type **Man-in-the-Middle**
- La modification des données en transit
- L'usurpation de session

Pour cela, un serveur **Nginx** a été déployé, jouant le rôle de **reverse proxy sécurisé**.

Installation de Nginx :

La première étape consiste à installer Nginx sur le serveur hébergeant la plateforme :

```
sudo apt update  
sudo apt install nginx
```

Nginx est ensuite utilisé pour exposer l'application Flask (port 5000) sur le port HTTPS (443).

Génération d'un certificat TLS auto-signé :

Dans le cadre d'un environnement pédagogique, un certificat **auto-signé** a été généré via OpenSSL :

```
sudo openssl req -x509 -nodes -days 365 \  
-newkey rsa:2048 \  
-keyout /etc/ssl/private/selfsigned.key \  
-out /etc/ssl/certs/selfsigned.crt \  
-subj "/CN=localhost"
```

Ce certificat assure :

- La **confidentialité** des échanges
- Le **chiffrement TLS**
- L'intégrité des données transportées

Bien que non validé par une autorité de certification, il est parfaitement adapté à un usage interne.

Configuration du reverse proxy Nginx :

Le fichier de configuration `/etc/nginx/sites-enabled/default` a été modifié pour activer :

- **Redirection automatique HTTP → HTTPS**

Cela empêche toute connexion non chiffrée :

```
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}
```

- **Proxy sécurisé vers l'application Flask**

L'application Flask reste accessible en interne sur 127.0.0.1:5000, mais n'est jamais exposée directement à Internet :

```
server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        proxy_pass http://127.0.0.1:5000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

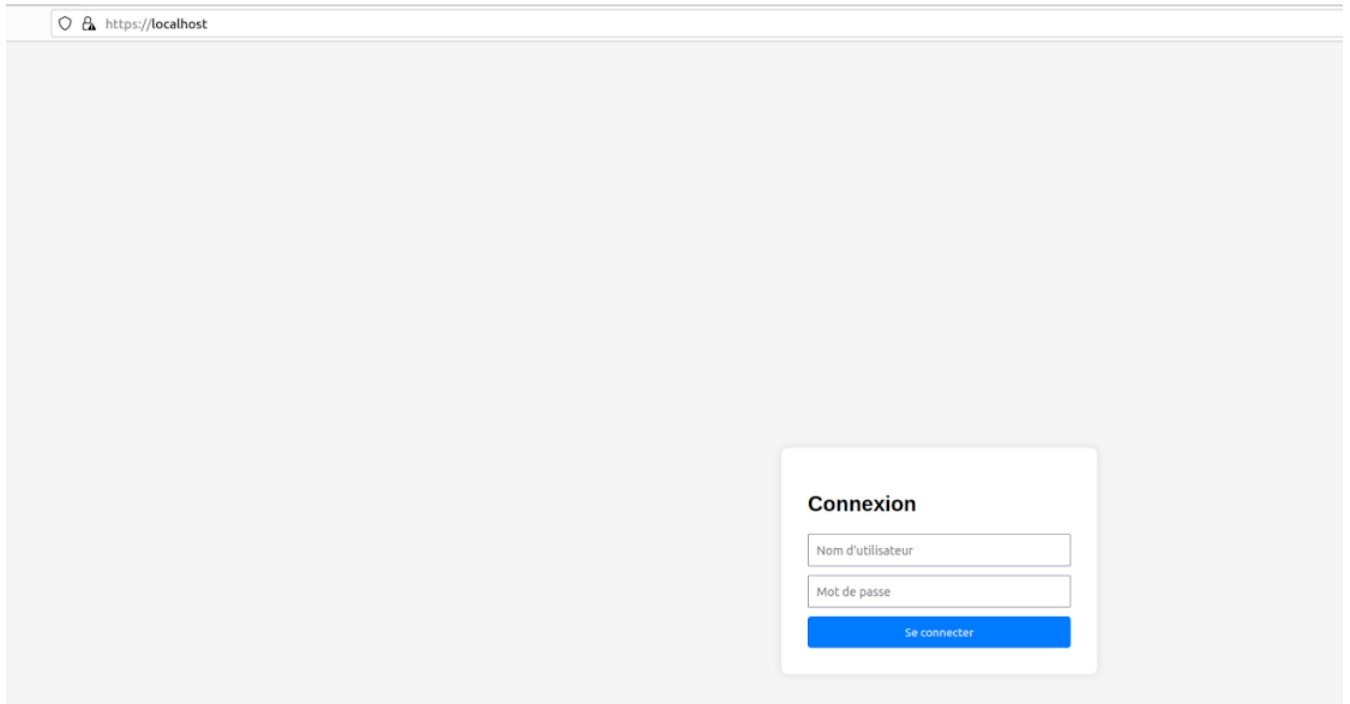
Cette configuration assure :

- La transmission correcte des informations client (IP, protocole...)
- La compatibilité avec Flask pour la gestion des sessions et redirections
- Un canal totalement chiffré pour toute requête utilisateur

Redémarrage du service

Après modification, Nginx a été redémarré pour appliquer la configuration :

```
sudo systemctl restart nginx
```



Procédures d'utilisation

Pour utiliser la plateforme, il est nécessaire de récupérer le projet depuis le dépôt GitHub. Le projet est disponible à l'adresse : <https://github.com/Imene-said/SAE501>. Il est possible de le cloner directement ou de le télécharger sous forme d'archive ZIP. Dans ce dernier cas, il suffit de dézipper l'archive dans un répertoire de votre choix.

Avant de lancer les conteneurs, il est indispensable d'installer les prérequis détaillés ([Voir page 6](#)), notamment Docker, Docker Compose et les dépendances nécessaires à la compilation des images. Une fois l'environnement prêt, il faut construire les images Docker correspondant aux différents TP.

Pour cela, rendez-vous dans le dossier SAE501/images-docker/<nom_image> et exécutez la commande suivante pour chaque conteneur :

```
docker build -t <nom_image>
```

Une fois toutes les images créées, retournez dans le dossier racine du projet SAE501 et lancez la plateforme avec **Docker Compose** :

```
sudo docker compose build  
sudo docker compose up
```

Ces commandes vont construire l'ensemble des services et démarrer les conteneurs dans le bon ordre, assurant ainsi une infrastructure stable et prête à l'usage pédagogique.

Pour arrêter la plateforme, ou libérer les ressources utilisées par les conteneurs, la commande suivante peut être utilisée à tout moment :

```
sudo docker compose down
```

Monitoring de l'admin

Le suivi et la supervision des conteneurs constituent un volet essentiel de notre plateforme, notamment pour garantir la bonne exécution des TP et détecter rapidement toute erreur de déploiement. Docker offre plusieurs outils et méthodes permettant d'observer l'état des conteneurs et d'identifier les dysfonctionnements éventuels.

Dans notre infrastructure pédagogique, nous utilisons principalement **Docker** et **Docker Compose** pour déployer les conteneurs des différents TP (Ubuntu Server, Ubuntu Desktop, Kali Linux, Développement ou encore personnalisé). Chaque conteneur peut être surveillé en temps réel via les commandes Docker (`docker ps`, `docker logs`, `docker inspect`) qui permettent de vérifier si un service s'exécute correctement, de consulter les ports exposés, et d'identifier les erreurs survenues lors du lancement ou du démarrage des applications internes, comme Gotty ou noVNC. Cette surveillance facilite la maintenance et le dépannage en cas de problèmes de configuration ou d'exécution.

Pour enrichir le suivi, nous avons intégré un **système de supervision des utilisateurs via Windows Server et RADIUS**. Chaque étudiant se connecte à la plateforme avec des identifiants gérés sur un serveur central, permettant d'associer les conteneurs lancés à un utilisateur précis. Cette configuration simplifie le contrôle de l'accès et offre un suivi précis des sessions, tout en renforçant la sécurité pédagogique : toute anomalie ou conteneur mal déployé peut être rapidement reliée à un utilisateur.

Enfin, la plateforme prévoit des indicateurs simples d'état pour chaque conteneur, tels que la disponibilité du port web, le statut du conteneur (démarrer, arrêter) et l'accessibilité des services VNC ou Web. Ces pratiques garantissent un monitoring suffisant pour un environnement éducatif : elles permettent d'identifier rapidement les conteneurs défaillants et de réagir avant que l'expérience pédagogique des étudiants ne soit impactée.

En résumé, la combinaison de **Docker**, des **logs conteneurs** et du **serveur Windows avec RADIUS** assure un suivi efficace, fiable et adapté à une infrastructure mononœud pédagogique, sans surcharger l'environnement d'outils trop complexes pour l'usage prévu.

Conclusion

La SAE 501 a permis de concevoir et de déployer une **infrastructure pédagogique moderne basée sur des conteneurs**. L'ensemble du projet s'appuie sur Docker et Docker Compose pour offrir un environnement flexible, reproductible et sécurisé, accessible via une interface web centralisée (Flask).

Points clés réalisés :

- **Déploiement et gestion d'environnements pédagogiques** : les utilisateurs peuvent créer et administrer des conteneurs Ubuntu, Kali Linux, Ubuntu Server et Ubuntu + Wireshark, avec un suivi précis via MariaDB
- **Sécurité et isolation** : chaque utilisateur est limité à ses propres conteneurs, l'authentification se fait via RADIUS, et les données sensibles sont protégées par chiffrement symétrique avec des clés stockées dans un fichier .env
- **Automatisation et administration** : l'interface Flask permet de déployer des conteneurs à la demande, de gérer les utilisateurs Windows via SSH et scripts PowerShell, et de suivre les actions dans la base de données
- **Utilisation d'outils modernes et adaptés** : Docker Compose v2 a été choisi pour sa simplicité, sa compatibilité avec l'infrastructure mononœud et ses fonctionnalités avancées, évitant la complexité de Kubernetes ou Docker Swarm
- **Sécurisation de l'accès** : HTTPS via Nginx, chiffrement des secrets, isolation des conteneurs, et gestion des ports dynamiques pour éviter les conflits

Cette SAE a permis de **réaliser une plateforme efficace et sécurisée**, démontrant la puissance de la virtualisation par conteneurs pour l'enseignement pratique. Les choix techniques ont été guidés par la simplicité, la compatibilité et la sécurité, tout en offrant aux utilisateurs une interface intuitive et un accès contrôlé à leurs environnements de TP. Cette solution démontre aussi qu'une infrastructure mononœud peut suffire pour un usage pédagogique tout en offrant des fonctionnalités avancées de déploiement et de gestion. Les choix techniques, tels que Docker Compose v2 plutôt que Kubernetes ou Docker Swarm, ont été guidés par la simplicité, la compatibilité et la maintenabilité de la plateforme.

Mots de passe utilisés

Pour chaque utilisateur : [Progtr00](#)

Mot de passe phpmyadmin : [root progtr00](#)

Mot de passe portainer : [Progtr000000](#)

Mot de passe vm ubuntu : [progtr00](#)

Mot de passe vm windows : [Progtr00](#)

Mot de passe vnc kali : [progtr00](#)

Mot de passe sudo kali : [kali](#)